

Homework 02 Data Handling, Graphics, More R

Due by 11:59pm, Friday 9.19.25

S&DS 2300/5300/ENV 757

(1) OK Cupid Dataset This is a dataset first discussed by Kirkegaard and Bjerrekaer (<https://openpsych.net/paper/46/>). We'll use this dataset several times this semester.

The data itself is stored as a parquet. This is a format where each column is stored separately (rather than as a giant array or matrix). It is more efficient to quickly access and search data. We will create the usual dataframe.

Run the code below to get the dataset itself and then to get a dataset that is the list of variables in the dataset.

```
#Run the line below one time only
install.packages("arrow")
library(arrow)
```

```
##
## Attaching package: 'arrow'
```

```
## The following object is masked from 'package:utils':
##
##      timestamp
```

```
#get the data - this takes a moment to load
okcupid <- data.frame(read_parquet("https://github.com/jreuning/sds230_data/raw/refs/heads/main/0KCupid/parsed_data_public.parquet", col_select = NULL, as_tibble = FALSE))

#get the dataset of questions
okcupidQ <- read.csv("https://raw.githubusercontent.com/jreuning/sds230_data/refs/heads/main/0KCupid/question_data.csv", sep = ";")
```

1.1) Write a few lines of code to get a sense of both the dataset and the variables it contains. Include whatever code you use below. Be sure to calculate the size of the dataset. For all code you write, always include a comment line on what you are doing.

As a part of this, you may want to open `okcupidQ` in the Environment (upper right quadrant of RStudio, click on Environment, double click on the dataset.) and browse through the questions.

Write code to get the unique values of `Type` in `okcupidQ`. What do you think these values mean?

Next, write not more than three sentences about what you observe about the dataset. Comment on missing values, the types of questions being asked, and whatever else you might notice.

```
# Get dimensions of the dataset
dim(okcupid)
```

```
## [1] 68371 2626
```

```
# Get names for okcupid that are not questions
col_names <- names(okcupid)
filtered_col_names <- col_names[!grepl("^q", col_names)]
filtered_col_names
```

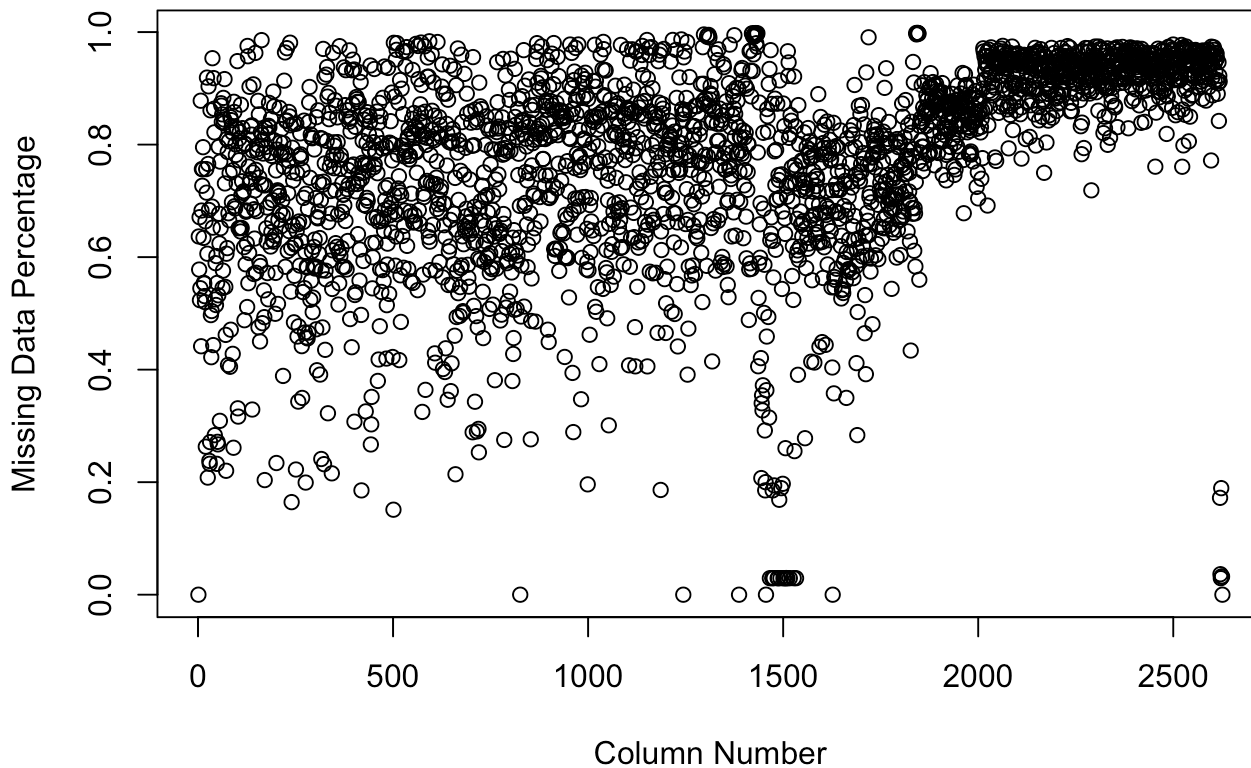
```
## [1] "Unnamed..0"      "p_conf"          "d_astrology_seriosity"
## [4] "p_laidback"      "d_education_phase" "p_drug"
## [7] "d_gender"        "p_lit"           "p_progress"
## [10] "d_income"        "p_roman"         "p_dominant"
## [13] "d_religion_type" "d_drugs"         "p_polit"
## [16] "p_pure"          "lf_want"         "d_smokes"
## [19] "p_manners"       "p_submissive"    "p_inde"
## [22] "p_kinky"         "p_opti"          "p_sloppy"
## [25] "p_indie"         "p_introvert"     "d_astrology_sign"
## [28] "d_country"       "p_arro"          "p_ambi"
## [31] "d_drinks"        "p_cool"          "p_trusting"
## [34] "d_age"           "p_thrift"        "d_ethnicity"
## [37] "p_organ"         "p_sexdrive"      "d_offspring_current"
## [40] "d_bodytype"      "p_honest"        "lf_max_age"
## [43] "lf_for"          "p_expsex"        "p_artsy"
## [46] "p_scien"         "d_job"           "d_orientation"
## [49] "d_languages"     "p_spon"          "p_explife"
## [52] "d_relationship"  "d_offspring_desires" "p_greed"
## [55] "p_capi"          "p_giving"        "p_explove"
## [58] "lf_location"     "p_convenmoral"   "p_aggre"
## [61] "p_kind"          "p_extro"         "p_friendstrangers"
## [64] "p_oldfash"       "p_comp"          "p_lovedri"
## [67] "p_sprit"         "lf_min_age"      "p_passion"
## [70] "d_education_type" "p_energetic"     "p_exer"
## [73] "p_logic"         "lf_single"       "p_math"
## [76] "d_religion_seriosity" "p_geeky"        "p_adven"
## [79] "CA"              "gender_orientation" "gender"
## [82] "race"            "gender2"         "gender2_num"
## [85] "CA_items"
```

```
# Get a small portion of the dataset
okcupid[1:5, 1:5]
```

```
##   Unnamed..0  q2      q11 q12  q13
## 1           1 <NA> Horrified <NA> <NA>
## 2           2 <NA>      <NA> <NA> <NA>
## 3           3 <NA>      <NA> <NA> No
## 4           4 <NA>      <NA> <NA> <NA>
## 5           5 <NA>      <NA> <NA> <NA>
```

```
# Explore how many NA's there are
percent_NA <- colSums(is.na(okcupid) / nrow(okcupid))
plot(percent_NA,
     ylab = "Missing Data Percentage",
     xlab = "Column Number",
     main = "Percent of Missing Data per Column")
```

Percent of Missing Data per Column



```
# Checking how many columns are empty for each person on average
mean(rowSums(is.na(okcupid)))
```

```
## [1] 2016.907
```

```
# Getting unique values of 'Keywords' in 'okcupidQ'
keywords <- strsplit(okcupidQ$Keywords, ";")
unique_keywords <- unique(trimws(unlist(keywords)))
unique_keywords
```

```
## [1] "sex/intimacy"      "preference"        "opinion"
## [4] "BDSM"              "religion/superstition" "descriptive"
## [7] "politics"          "cognitive"         "technology"
```

```
# Getting unique values of 'Type' in 'okcupidQ'
unique(okcupidQ$Type)
```

```
## [1] "N" "O" "M" ""
```

```
# Seeing how many times each unique value of 'Type' in 'okcupidQ' shows up
table(okcupidQ$Type)
```

```
##
##           M      N      O
##    81   154   320 2065
```

The unique values of 'Type' in 'okcupidQ' include "N", "M", "O", and "". It generally looks like "" is used for rows in 'okcupidQ' to denote a row that does not ask a question, but rather contains other statistics. I think "N" and "O" stand for nominal and ordinal. As for "M", I don't know if there is a word that succinctly describes it, but it seems to be used for questions whose answer options do not have similar responses.

When it comes to the dataset, I thought it was interesting that there were so many empty cells. I assumed that there would be a somewhat even distribution of these empty cells if questions were answered randomly, but based on the plot, it seems there's a large uptick in missing responses when we get around column 1900. Lastly, the unique keywords make sense given that okcupid is a dating site, explaining the existence of categories like sex/intimacy and BDSM.

1.2) Write exactly three questions that you are curious to answer using this dataset.

1. How does having tattoos correlate to whether or not someone is a virgin?
2. How important is each response when it comes to matchmaking?
3. Are there any common responses in the dataset that differs from populations that do not use dating apps?

1.3) We are going to calculate a numeric value of income. I did this in the code below.

```
grep("income", names(okcupid))
```

```
## [1] 1469
```

```
okcupid[, 1469][1:100]
```

##	[1]	NA	NA	NA
##	[4]	NA	NA	NA
##	[7]	NA	NA	NA
##	[10]	NA	NA	NA
##	[13]	NA	NA	NA
##	[16]	"\$20,000-\$30,000"	NA	NA
##	[19]	NA	NA	NA
##	[22]	NA	"\$50,000-\$60,000"	NA
##	[25]	NA	NA	NA
##	[28]	NA	NA	NA
##	[31]	"\$30,000-\$40,000"	"\$40,000-\$50,000"	NA
##	[34]	NA	NA	NA
##	[37]	NA	NA	NA
##	[40]	NA	NA	NA
##	[43]	NA	"\$20,000-\$30,000"	"\$80,000-\$100,000"
##	[46]	NA	"Rather not say"	NA
##	[49]	NA	NA	NA
##	[52]	NA	NA	NA
##	[55]	NA	NA	NA
##	[58]	NA	NA	"Less than \$20,000"
##	[61]	NA	NA	NA
##	[64]	NA	NA	NA
##	[67]	NA	NA	NA
##	[70]	NA	NA	NA
##	[73]	NA	NA	NA
##	[76]	NA	NA	NA
##	[79]	NA	NA	NA
##	[82]	NA	NA	NA
##	[85]	NA	"Rather not say"	NA
##	[88]	NA	NA	"Rather not say"
##	[91]	NA	"\$30,000-\$40,000"	NA
##	[94]	"Rather not say"	NA	NA
##	[97]	NA	NA	NA
##	[100]	NA		

```
names(okcupid)[1469]
```

```
## [1] "d_income"
```

```
income <- okcupid[, 1469]
unique(income)
```

##	[1]	NA	"\$20,000-\$30,000"	"\$50,000-\$60,000"
##	[4]	"\$30,000-\$40,000"	"\$40,000-\$50,000"	"\$80,000-\$100,000"
##	[7]	"Rather not say"	"Less than \$20,000"	"\$60,000-\$70,000"
##	[10]	"More than \$1,000,000"	"\$100,000-\$150,000"	"\$70,000-\$80,000"
##	[13]	"\$250,000-\$500,000"	"\$150,000-\$250,000"	"\$500,000-\$1,000,000"

```

parse_income <- function(x) {
  if (is.na(x) || x == "Rather not say") {
    return(NA_real_)
  } else if (x == "More than $1,000,000") {
    return(1000000)
  } else if (x == "Less than $20,000") {
    return(20000)
  } else if (grepl("-", x)) {
    nums <- gsub("[\\$,]", "", x)
    parts <- strsplit(nums, "-")[1]
    return(mean(as.numeric(parts)))
  }
  return(NA_real_)
}

okcupid$income_numeric <- as.numeric(sapply(income, parse_income))

```

Admittedly, much of the code above is things we have not yet discussed.

Run the code to see what it does. You may want to run individual lines or subsections. Feel free to ask AI specific questions about functions.

Your main job is to add comments to the code to describe what each part does. Furthermore, in your comments indicate the thought process behind the code (i.e. what question prompted the code?).

Putting the code into AI will do this for you - that said, you'll ultimately want to make sure that if I or others gave you the code without comments that you could read the code for yourself.

Finally - make any plot you like to examine the distribution of income and discuss what you observe.

```

# Returns which column(s) have "income" in the okcupid dataset. Used to narrow down which columns we'd like to possibly look at when calculating income
grep("income", names(okcupid))

```

```
## [1] 1469 2627
```

```

# Returns the entries for 100 respondants for a column which we believe is income. Used to get a feel for how the data is formatted and how much missing data there is.
okcupid[, 1469][1:100]

```

##	[1]	NA	NA	NA
##	[4]	NA	NA	NA
##	[7]	NA	NA	NA
##	[10]	NA	NA	NA
##	[13]	NA	NA	NA
##	[16]	"\$20,000-\$30,000"	NA	NA
##	[19]	NA	NA	NA
##	[22]	NA	"\$50,000-\$60,000"	NA
##	[25]	NA	NA	NA
##	[28]	NA	NA	NA
##	[31]	"\$30,000-\$40,000"	"\$40,000-\$50,000"	NA
##	[34]	NA	NA	NA
##	[37]	NA	NA	NA
##	[40]	NA	NA	NA
##	[43]	NA	"\$20,000-\$30,000"	"\$80,000-\$100,000"
##	[46]	NA	"Rather not say"	NA
##	[49]	NA	NA	NA
##	[52]	NA	NA	NA
##	[55]	NA	NA	NA
##	[58]	NA	NA	"Less than \$20,000"
##	[61]	NA	NA	NA
##	[64]	NA	NA	NA
##	[67]	NA	NA	NA
##	[70]	NA	NA	NA
##	[73]	NA	NA	NA
##	[76]	NA	NA	NA
##	[79]	NA	NA	NA
##	[82]	NA	NA	NA
##	[85]	NA	"Rather not say"	NA
##	[88]	NA	NA	"Rather not say"
##	[91]	NA	"\$30,000-\$40,000"	NA
##	[94]	"Rather not say"	NA	NA
##	[97]	NA	NA	NA
##	[100]	NA		

Returns the full name of the column we believe is income. Used to double check if we got the income column and not a random column that has income in there (hourly income, income tax, etc.).

```
names(okcupid)[1469]
```

```
## [1] "d_income"
```

Makes a vector of values from okcupid's "d_income" column. Used for future data transformation.

```
income <- okcupid[, 1469]
```

Finds the unique values in our income vector. Used to understand what values we want to transform

```
unique(income)
```

##	[1]	NA	"\$20,000-\$30,000"	"\$50,000-\$60,000"
##	[4]	"\$30,000-\$40,000"	"\$40,000-\$50,000"	"\$80,000-\$100,000"
##	[7]	"Rather not say"	"Less than \$20,000"	"\$60,000-\$70,000"
##	[10]	"More than \$1,000,000"	"\$100,000-\$150,000"	"\$70,000-\$80,000"
##	[13]	"\$250,000-\$500,000"	"\$150,000-\$250,000"	"\$500,000-\$1,000,000"

```

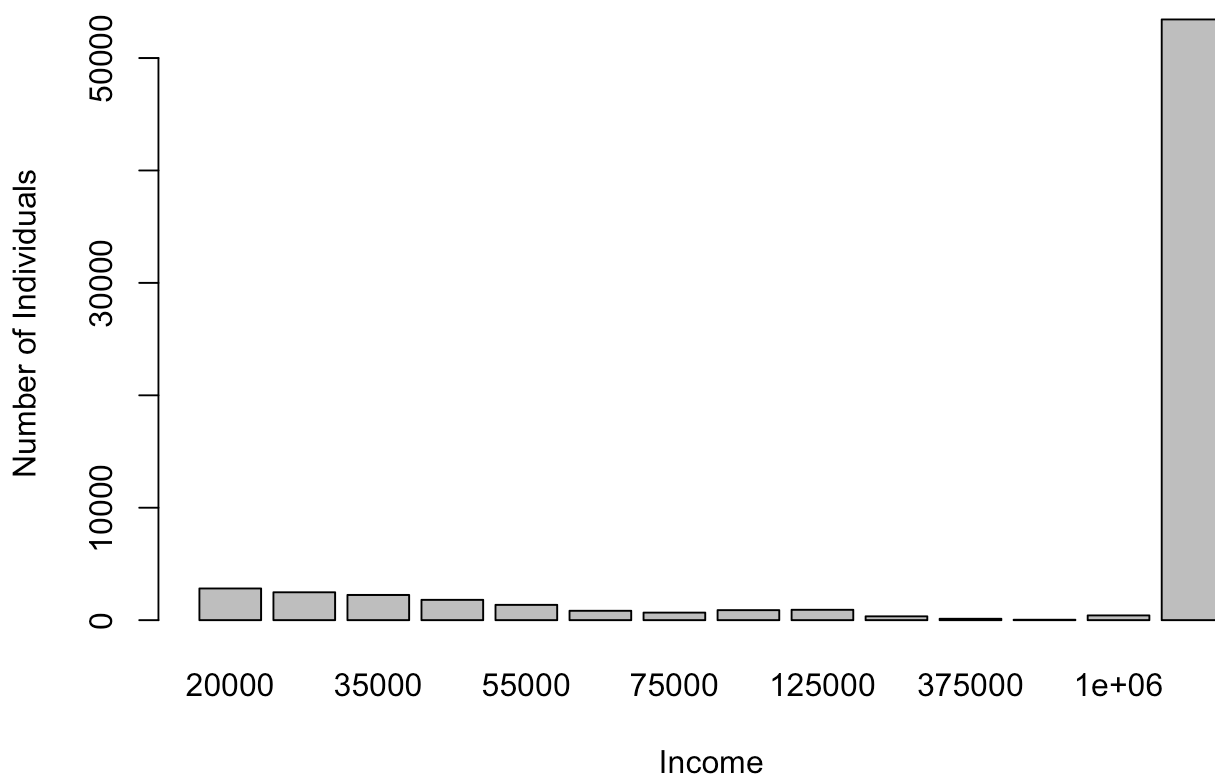
# This function we are making is used to convert our values from the income column to numeric income values. Can be used to do work that requires numeric values, such as some types of plots.
parse_income <- function(x) {
  if (is.na(x) || x == "Rather not say") {
    # Checks to see if the income value is empty/"Rather not say". If true, returns a numeric version of NA
    return(NA_real_)
  } else if (x == "More than $1,000,000") {
    # Checks to see if the income value is "More than $1,000,000". If true, returns 1000000
    return(1000000)
  } else if (x == "Less than $20,000") {
    # Checks to see if the income value is "Less than $20,000". If true, returns 20000
    return(20000)
  } else if (grepl("-", x)) {
    # Checks to see if the income value is within a range. If true, returns the numeric mean of the range
    nums <- gsub("[\\$,]", "", x)
    parts <- strsplit(nums, "-")[1]
    return(mean(as.numeric(parts)))
  }
  # If the income value does not match any of the if statements above, then returns the numeric version of NA
  return(NA_real_)
}

# Makes a new column in okcupid names income_numeric and fills it by using the parse_income function on the income values.
okcupid$income_numeric <- as.numeric(sapply(income, parse_income))

# Make a barplot to see the frequency of each numeric income value in the okcupid dataset
barplot(table(okcupid$income_numeric, useNA = "ifany"),
        xlab = "Income",
        ylab = "Number of Individuals",
        main = "Income Frequency in okcupid Dataset (Including NA)")

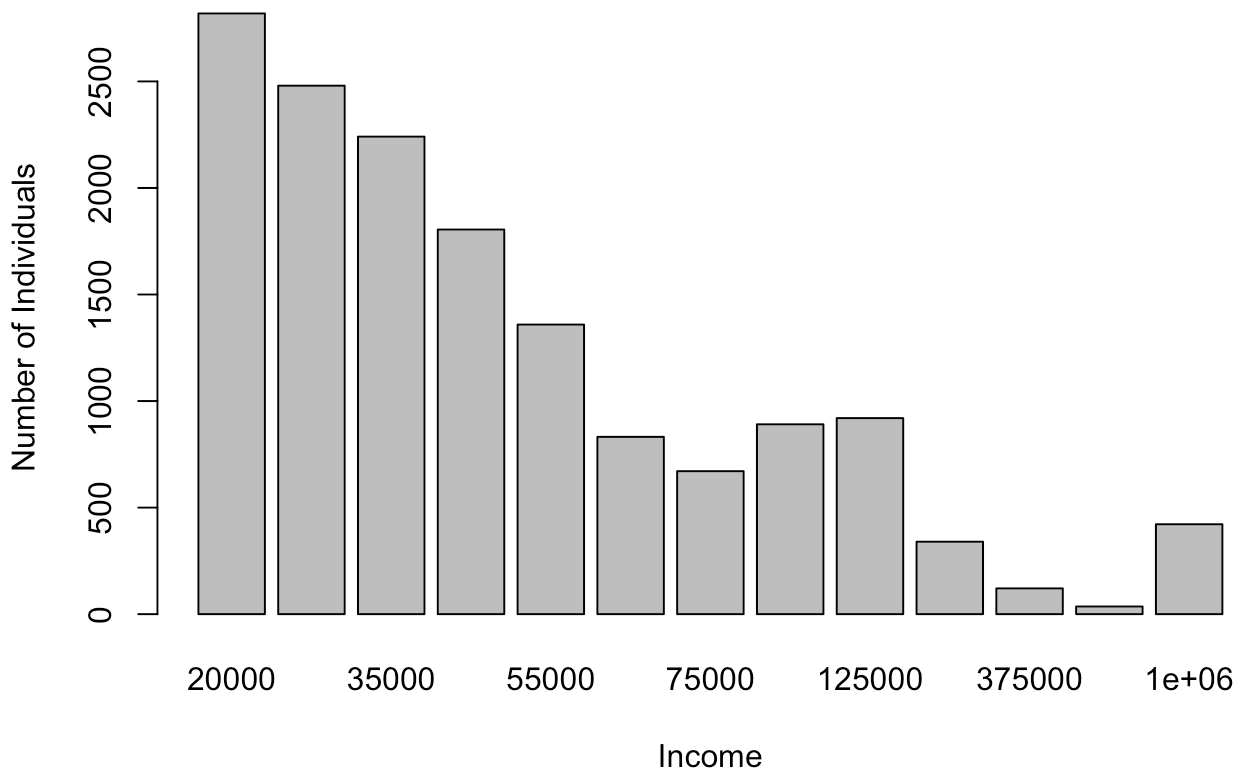
```


Income Frequency in okcupid Dataset (Including NA)



```
# Make the same bar plot as before, but remove NA to get a clearer view of the results
barplot(table(okcupid$income_numeric),
        xlab = "Income",
        ylab = "Number of Individuals",
        main = "Income Frequency in okcupid Dataset (Excluding NA)")
```

Income Frequency in okcupid Dataset (Excluding NA)



In the plots above, we can see that there is a general downward trend in number of individuals as income increases. However, because so many people did not answer this question, the distribution can be very different in reality.

(1.4) Pick any variable EXCEPT q30458 that starts with q that has four possible outcomes (options).

- Create and display a table called `table1` that shows how many of instances there were of each level.
- Write a line that calculates the percentage of values that were from the fourth level of the question, rounded to the nearest whole percent.
- Write a line of code to use `paste()` to make a statement like “x% of respondents responded (response) to question (text of the question pulled from okcupidQ).”

```
# Create and display 'table1' that shows how many of instances there were of each level
table1 <- table(okcupid$q77)
table1
```

```
##
##      Never      Rarely  Sometimes Very often
##      3980      11220      31149      6118
```

```
# Calculates percentage of values from the fourth level of the question rounded to the nearest whole percent
alc_percents <- round(table1 / sum(table1) * 100, 0)
alc_percent_never <- alc_percents[1]
alc_percent_never
```

```
## Never
##      8
```

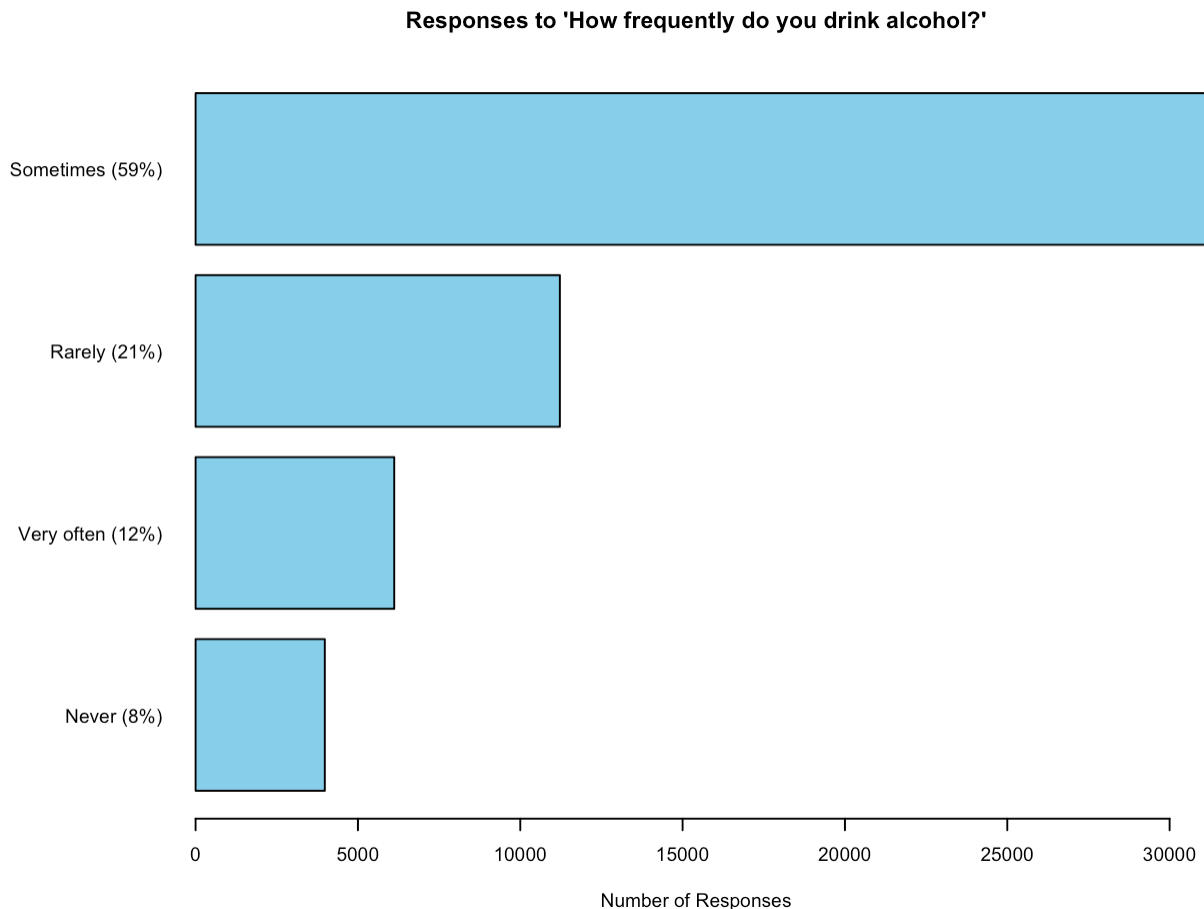
```
# paste() to make a statement like "x% of respondents responded (response) to question"
paste(alc_percent_never, "% of respondents responded '", names(alc_percent_never), "' to '", okcup
idQ$text[55], "'", sep = "")
```

```
## [1] "8% of respondents responded 'Never' to 'How frequently do you drink alcohol?'"
```

(1.5) Create a barplot using the `barplot()` function for whatever question you chose in part (1.4).

- The labels of the barplot should also contain the whole number percentages for each possible outcome.
- Format your graph, and make sure the bars are horizontal.
- Change the margins using `par(mar = ...)` if your labels are being cut off, and change the character size if necessary.

```
if(!grepl("%", names(table1)[1])){
  names(table1) <- paste0(names(table1), " (", alc_perents, "%)")
}
par(mar = c(5, 12, 4, 2), cex = 0.6)
barplot(sort(table1),
        horiz = T,
        las = 1,
        col = "skyblue",
        main = "Responses to 'How frequently do you drink alcohol?'",
        xlab = "Number of Responses")
```



(1.6) Choose any variable that has a Yes/No response.

- Make a two-way table between this new question and your question in 1.4.
- Calculate the percent of people for each level of your 1.4 question that answered 'Yes' to your new question.

- Following the example in Class 4, create a barplot akin to that in lines 431-442 with similar percentages displayed. This will take some adjusting. You can make horizontal or vertical. Make sure no labels are cut off.
- Does it seem like these variables might be related?

```
# A two-way table between q501 and q77
two_table <- table(okcupid$q501, okcupid$q77)
two_table
```

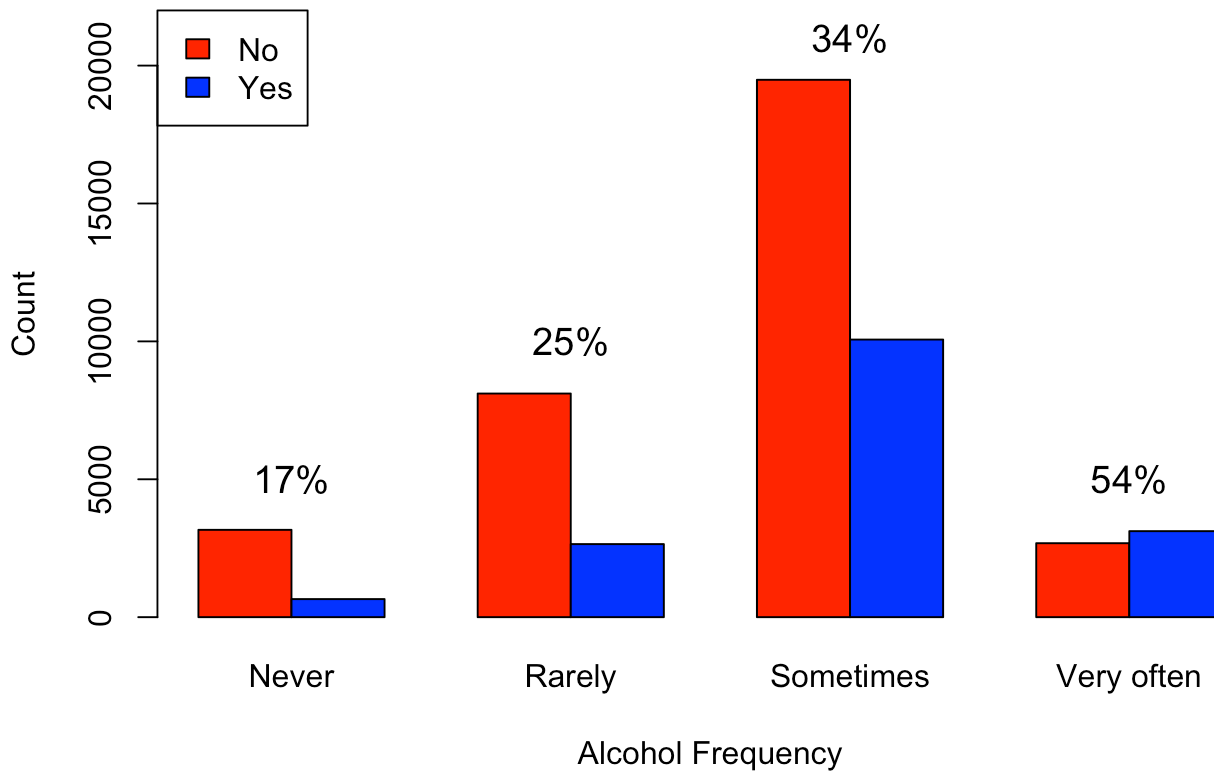
```
##
##      Never Rarely Sometimes Very often
## No    3166    8109      19485      2682
## Yes    655    2649      10066      3119
```

```
# Calculate the percent of people for each level of your 1.4 question that answered 'Yes' to your
new question
yes_counts <- two_table["Yes", ]
col_counts <- colSums(two_table)
yes_percent <- paste(round(yes_counts / col_counts * 100, 0), "%", sep = "")
yes_percent
```

```
## [1] "17%" "25%" "34%" "54%"
```

```
# Barplot akin to example in Class 4 (lines 431-442)
barplot(two_table,
        col = c("red", "blue"),
        beside = T,
        legend = T,
        args.legend = list(x = "topleft"),
        main = "Recent Smoking by Alcohol Consumption Frequency",
        xlab = "Alcohol Frequency",
        ylab = "Count",
        ylim = c(0, 22000))
text(c(2,5,8,11), c(5000,10000,21000,5000), yes_percent, cex = 1.2)
```

Recent Smoking by Alcohol Consumption Frequency



These variables seem very related, as the percentage of recent smoking consistently increases as alcohol frequency increases. This would make sense if we assume that there's a high correlation between different types of substance abuse.

(1.7) Choose any two variables that start with the letter `p`. These are questions that are calculated scales.

- Calculate summary statistics using `summary()` and also the standard deviation for each variable (rounded to 2 decimal places).
- Make histograms for each variable. Format your histogram.
- Describe the shape of each histogram (use words like 'symmetric' or 'skewed') and discuss what you observe.

```
# Summary statistics and standard deviation for "Into exercise"
summary(okcupid$p_exer)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NA's
## -94.00  -30.00   -3.00    1.71  40.00   97.00   49948
```

```
round(sd(okcupid$p_exer, na.rm = TRUE), 2)
```

```
## [1] 45.43
```

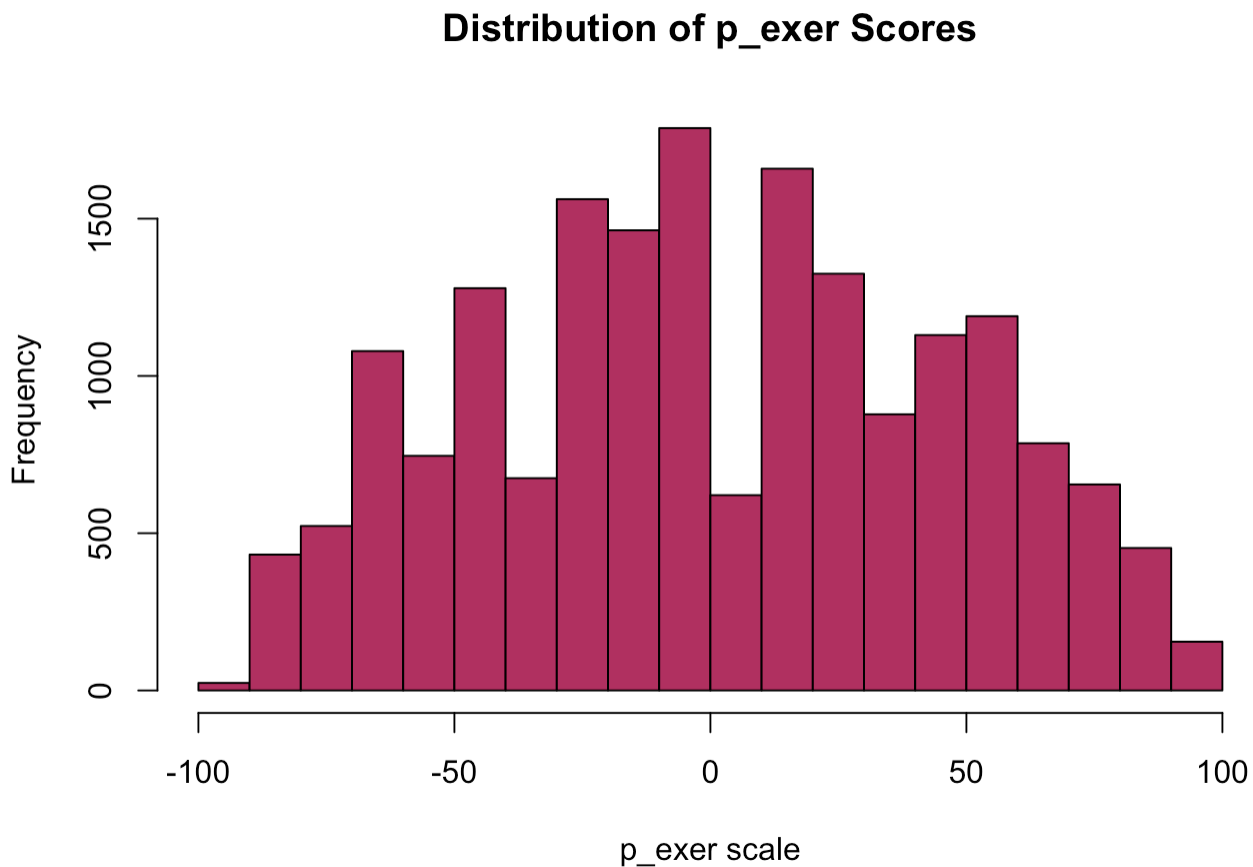
```
# Summary statistics and standard deviation for "Spontaneous"
summary(okcupid$p_spon)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NA's
## -92.00  -20.00   10.00   10.63  45.00   99.00   49648
```

```
round(sd(okcupid$p_spon, na.rm = TRUE), 2)
```

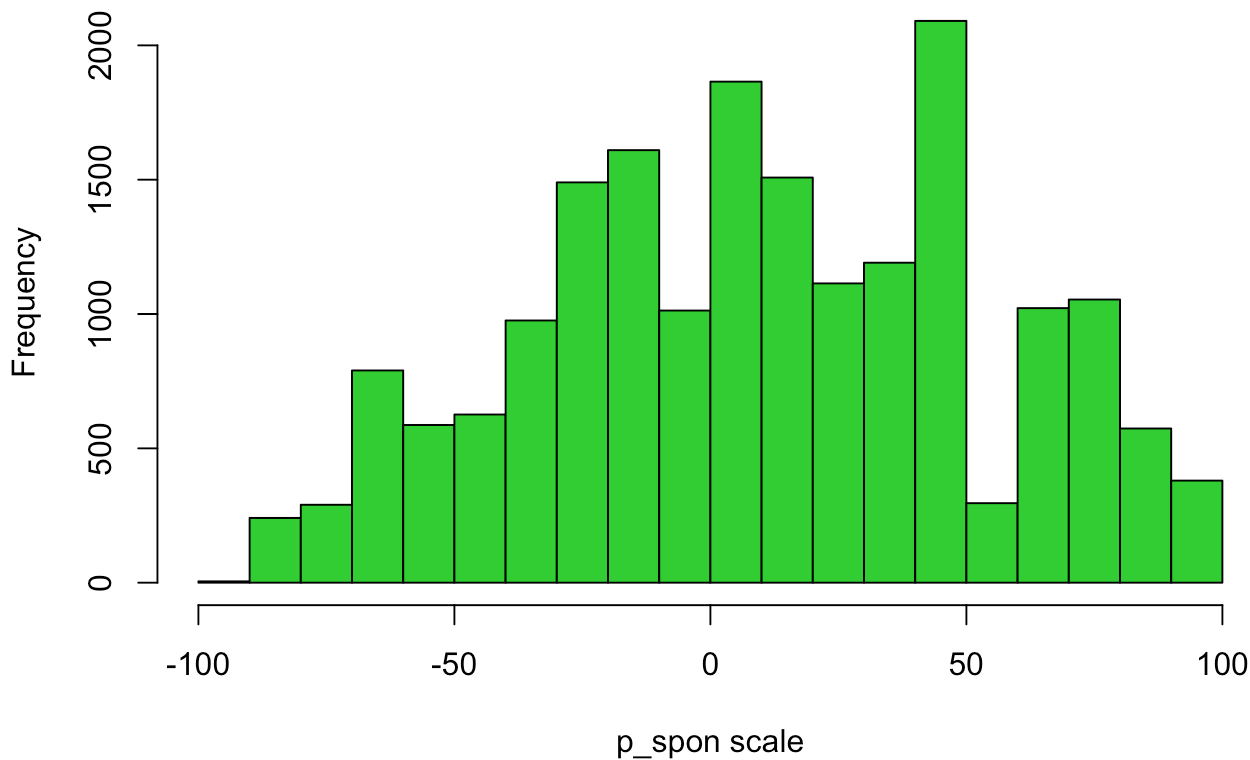
```
## [1] 43.59
```

```
# "Into exercise" histogram  
hist(okcupid$p_exer,  
      main = "Distribution of p_exer Scores",  
      xlab = "p_exer scale",  
      col = "maroon")
```



```
# "Spontaneous" histogram  
hist(okcupid$p_spon,  
      main = "Distribution of p_spon Scores",  
      xlab = "p_spon scale",  
      col = "limegreen")
```

Distribution of p_spon Scores



The histogram for *p_exer* generally looks quite symmetrical. Similarly, *p_spon* also looks quite symmetrical, albeit a tiny bit skewed to the right, seemingly indicating that more people are calculated to be spontaneous.

(1.8) Choose either of the variables you used in 1.7 AND the Yes/No variable you choose in 1.6.

- Get summary statistics for the *p_* variable by each level of your Yes/No variable. Compare the medians and means of these two distributions - what do you observe?

```
summary(okcupid$p_exer[okcupid$q501 == "Yes"])
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.   NA's
## -91.000 -40.000   -9.000   -6.308  27.000   96.000  25272
```

```
summary(okcupid$p_exer[okcupid$q501 == "No"])
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.   NA's
## -94.00  -29.00    7.00    4.97  43.00   97.00  36943
```

Both the median and mean are higher for *p_exer* when people have not smoked within the last 6 months. This makes sense since people that are into exercising for health reasons would probably not want to smoke, as that would be a detriment to their health.

(1.9) Make a scatterplot of your two *p_* variables from from 1.6).

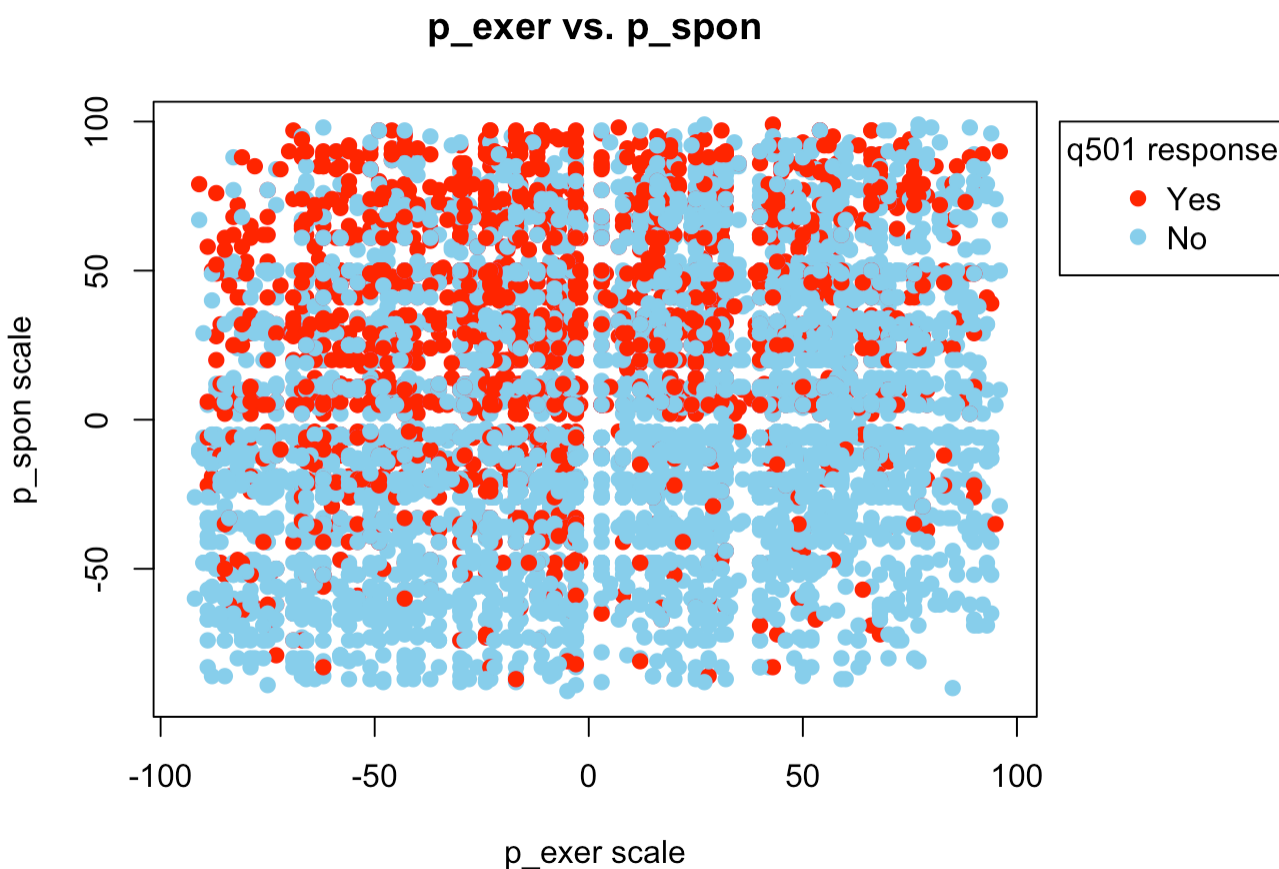
- Set a non-default plot character.
- Create the plot using different colors for each level of your binary variable from 1.7.
- Add a legend to your plot

- Format and label your plot
- Discuss any trends you see as well as any differences between groups.

```
par(mar = c(5, 4, 4, 8))

plot(okcupid$p_exer, okcupid$p_spon,
     main = "p_exer vs. p_spon",
     xlab = "p_exer scale",
     ylab = "p_spon scale",
     # "Yes" = "Red", "No" = "SkyBlue" because of the negative effects of smoking and positive ef
     # fects of not smoking
     col = c("Yes" = "Red", "No" = "SkyBlue")[okcupid$q501],
     pch = 19)

legend(x = 110,
      y = 100,
      legend = c("Yes", "No"),
      col = c("red", "skyblue"),
      pch = 19,
      title = "q501 response",
      xpd = TRUE)
```



It looks like there's a trend that people who have smoked a cigarette in the last 6 months are also more spontaneous. The previous trend of recent smoking having a lower p_exer value still remains true. This combination of trends make it so that, as you go from top left to bottom right, you are less likely to have smoked a cigarette in the last 6 months.

(1.10) Create a subset of the original dataset that meets the following conditions:

- It has complete data for all five of the variables you have examined so far (income_num, your four level variable, your two p_ indices, and your binary variable) as well as the variable d_astrolology_sign

- You only want people who have Ca or Sa as the first two letters of their astrological sign.

Use both the `%in%` operator and the `substr()` function to complete this task. You'll need to look up information about how these work.

Get the dimension of your final dataset. Comment on the reduction in size from the original dataset.

```
okcupid_subset <- okcupid[, c("income_numeric", "q77", "p_exer", "p_spon", "q501", "d_astrology_sign")]
okcupid_subset <- okcupid_subset[complete.cases(okcupid_subset), ]
okcupid_subset <- okcupid_subset[substr(okcupid_subset$d_astrology_sign, 1, 2) %in% c("Ca", "Sa"), ]
dim(okcupid_subset)
```

```
## [1] 521 6
```

The size of the subset is considerably smaller than the original. Before, we had 2626 columns and now we have 6. For rows, we had 68371 rows and now we have 521. I think a lot of the row reduction comes from removing rows that had incomplete responses, as we would have a lot more than just 521 rows from subsetting based only on astrology sign.

(1.11) For any one of the questions you wrote in 1.2), investigate this BRIEFLY with some combination of a plot and basic summary statistics (or a table).

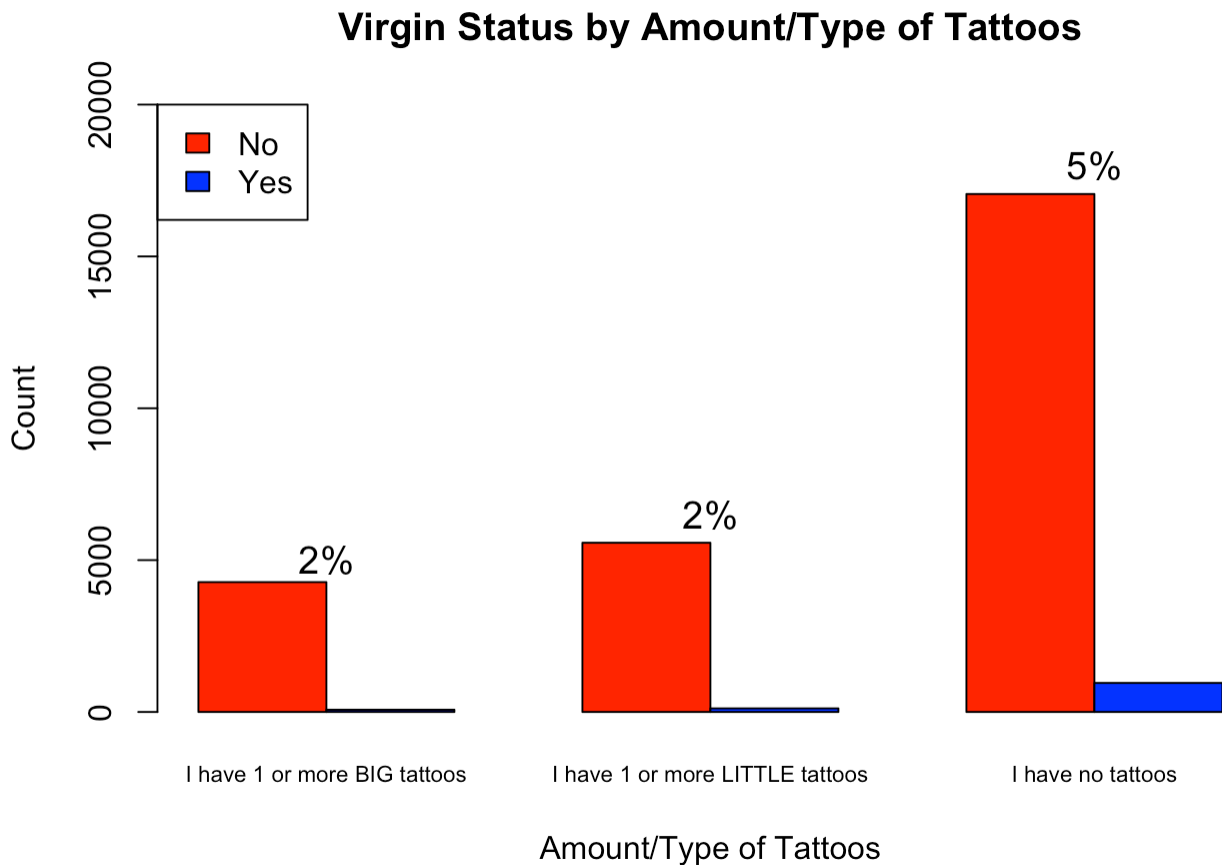
```
#q275 is virgin status and q128 is how many tattoos you have
tattoo_virgin_table <- table(okcupid$q275, okcupid$q128)
tattoo_virgin_table
```

```
##
##      I have 1 or more BIG tattoos I have 1 or more LITTLE tattoos
## No                        4276                        5572
## Yes                        73                          118
##
##      I have no tattoos
## No                        17055
## Yes                        956
```

```
# Calculate the percent of people for each level of q128 that answered 'Yes' to your new question
yes_counts_t_v_table <- tattoo_virgin_table["Yes", ]
col_counts_t_v_table <- colSums(tattoo_virgin_table)
yes_percent_t_v_table <- paste(round(yes_counts_t_v_table / col_counts_t_v_table * 100, 0), "%",
sep = "")
yes_percent_t_v_table
```

```
## [1] "2%" "2%" "5%"
```

```
# Make a barplot to visualize tattoos vs virgin status
barplot(tattoo_virgin_table,
        col = c("red","blue"),
        beside = T,
        legend = T,
        args.legend = list(x = "topleft"),
        main = "Virgin Status by Amount/Type of Tattoos",
        xlab = "Amount/Type of Tattoos",
        cex.names = 0.7,
        ylab = "Count",
        ylim = c(0, 20000))
text(c(2,5,8), c(5000,6500,18000), yes_percent_t_v_table, cex = 1.2)
```



The question I chose to investigate was “How does having tattoos correlate to whether or not someone is a virgin?”. I made a two way table showing the counts for the data and then made bar plots similar to question 1.6. Here, we can see that there is a small difference in the percentages and that the data has a LOT more people who are not virgins than are.

(2.1) Your work at a social research nonprofit puts you in touch with a large, multi-year survey called the National Civic Life Survey. Each row is one respondent; columns include demographics (age, education), civic behaviors (voting frequency, volunteering hours), and attitude questions. Many questions were optional, so missing values are common. The data arrive as a Parquet file, plus a small CSV with question metadata.

Using only the tools you’ve learned so far, write out in words the steps you would take to explore this dataset and generate a few simple plots. Do not include any R code, but you’re welcome to mention R functions you might use.

Hint: In this HW, you worked with parquet/CSV files, inspected structure and summaries, examined unique values for categorical variables, and noted missingness.

- Load in the Parquet file and question metadata CSV as dataframes
- Get dimensions and column names of the dataset

- In addition to this, I'd identify any columns that were not questions
- Find how many cells were empty, both via rows and columns
- Get column names of the dataset
 - If there are things like type or keyword, use `unique()` to find what we're working with
- Create new columns in the dataset for columns that can logically be turned into a numeric value
 - For example, if volunteering hours is split into hours categories as characters, take the mean or median of each category and make a new column in numeric form
- Optional: If I wanted to do analysis that would want rows/columns that have few NAs, I would make a subset to exclude any rows/columns with a lot of NAs.
- Make a histogram of age
 - This would allow us to view the age distribution of the dataset (Is it normal? do all respondents need to be adults? etc.)
- Make a bar plot for voting frequency vs age
 - In the bar plot, I'd put age on the x axis and voting frequency on the y axis. If age is a numeric number, I'd group them into broad age groups (for example, we can differentiate via generation).
 - If we had data on what political party each individual is registered with, we can have that reflected in the bar plot and put a legend on the plot.
- Make a scatterplot for age vs volunteering hours
 - We can use `summary()` to get a numerical idea of how age and volunteering hours is distributed in the dataset.
 - We can also color the points via education and put a legend in the corner/outside the box.